

Computing Science (CMPUT) 455

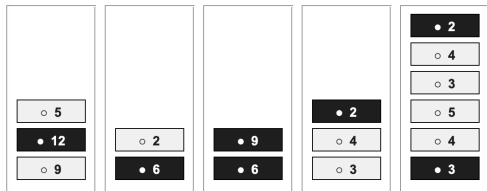
Search, Knowledge, and Simulations

Martin Müller

Department of Computing Science
University of Alberta
`mmueller@ualberta.ca`

Fall 2026

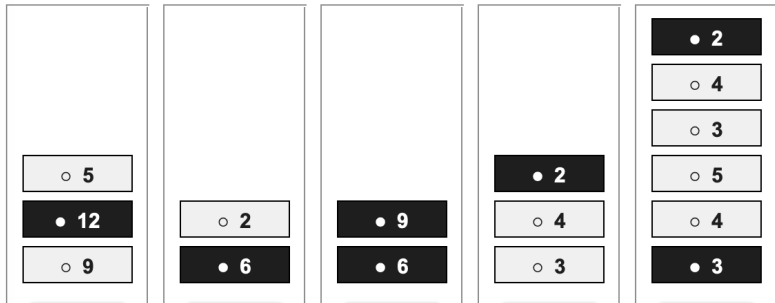
Assignment 1 Preview - Heap Go



Assignment

- Implement rules and a random player for **Heap Go**
- Heap Go is a two player game modeling (very) simplified Go endgames
- Use our starter code `a1.py`
- Implement a number of *text commands*

Heap Go



- Now: in-class demo

Assignment 1 To Do, and Learning Goals

Do now:

- Form your team, and inform the designated TA (latest: Sep 9)
- Read the assignment spec on Canvas
- Understand the starter code

Learning Goals

- Become familiar with coding a simple game in Python
- Implement Heap Go rules and a random player
- Implement a text-based interface for a game
- Prepare for future assignments

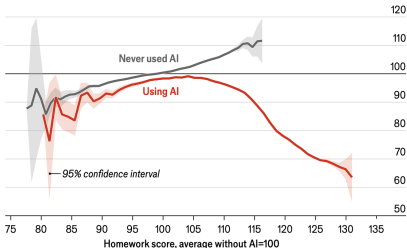
Homework and AI Use

- Optional: use AI coding assistant
 - To help *review and improve* your code
 - To **help you learn** concepts, Python details, ...
- DO NOT let AI write your code for “easy marks”
- Short term “gain”, long term loss...
- See high school study: homework vs exam score

Thinking about it

China*, average exam score

January 2023-June 2025, average without AI=100



*26,811 students aged 12-18

Source: "The generative AI learning penalty: evidence from Chinese secondary education",
by D. Strömberg et al., preprint June 2026

Assignment 1, Part A - Implement Rules of Heap Go

Implement rules:

- Create a starting position = initial game state
- Check: Is the game over?
- If not over: Play a move and update the game state
- What are the legal moves?
- implement text commands related to rules and game state:
`heapgo, show, toplay, play, legal, genmove, score, winner`
- All the details: read the Assignment 1 specification

Assignment 1, Part B: Random Player for Heap Go

For each move, your computer player should:

- Create a list of all legal moves
- Choose one move to play randomly from the list
- Recognize the end of the game and determine the winner
- Extend `a1.py` **sample code**
 - It implements a framework for text commands
 - Otherwise, it provides structure: mostly empty placeholder functions
 - In your assignment you implement these functions
- All the details: read the Assignment 1 specification

Assignment 1 Starter Code

- Download `a1.tgz` from assignment page
- Contains an `a1` directory, including file `a1.py` for you to modify
- The code has some extra comments and placeholders for the assignment
- Also contains **public tests** for the assignment
- You should write **many more** of your own tests
- We will test many more details, corner cases in **our private tests**

Assignments - Lessons Learned in Previous Years

- Lesson #1: Testing is Key
- Most programmers do **not test enough**
- Test everything
- Test after each change
- Use the scripts and test cases we provide
- Write your own tests
- Re-check the spec against what your program does

Test your Submission Before you Submit

- Do the same steps that we will do
- Go to (or login to) a lab machine in U-COMM
- Unpack your candidate final submission `a1.tgz` in a **new directory**
- Check if everything matches our specification: names, directory structure, contents
- Run the public test cases with this code
- Run your own test cases with this code
- Submit your `a1.tgz` if everything is OK

Assignments - More Lessons Learned

- Lesson #2: manage your time
- Time is limited - you need to manage it
- If you're in a team - teams need time to communicate
- Our experience: No matter what the deadline, and how much time there is, the number of questions *grows exponentially* in the last two days
- The last two days are the
worst possible time
to do your homework. Stress for all.
- Pace yourself. Manage your time. Do things early.
 - It will be **seriously good for you** (and us).

Lessons Learned - Polya's Principles

- We will later discuss Polya's book *How to Solve It*
- Polya's first principle: **Understand the Problem**
- Read the spec. Implement it. Completely and precisely.
- Then read again and **review your solution**
(Polya's fourth principle)

More Advice

- If you do not completely understand all of the problem:
 - Ask for clarification
 - The sooner the better
 - Use the Canvas forum. Use office hours.
- To really understand details and possible obstacles:
 - You need to engage and work with the material
 - Just reading the spec will not be enough
 - There will be related questions in quizzes and exams
 - You will not learn by *just* using an AI coding assistant - use it as a tool, not to replace you
- Time management -
engage early to leave enough time for these activities